LESSON PLAN · GENERATIVE AI (GR 6–8) INTERMEDIATE: GEN-AI CREATORS · LEVEL 2

Lesson 4: Project: AI-Assisted Mini Game

Module 1 | 60 minutes | You code, AI coaches | Grades 7–11 · ages 12–16

Objective: Plan, code and test your own mini game, using AI only to explain and debug, and keep an honest log of exactly what the AI did.

Success criteria

- I can plan and build a small game with AI as a coach.
- I can keep an honest what-AI-did log.
- I can test my game with five player tests.
- I can explain every line of my code.

Key vocabulary

- **mini game**: A small game with a clear goal and ending.
- **what-AI-did log**: A record of each time you used AI: asked, said, checked, result.
- **player test**: Playing your game with chosen inputs to check it works.
- **random.randint(a, b)**: Python: a random whole number from a to b, both included.
- **ownership**: Being able to explain and stand behind every part of your work.

You will need

- Projector or screen share with Kitabash and the AI tools for this lesson
- Devices: 1 per learner (laptop, tablet or phone)
- Learner accounts only where allowed: see the course safety notes
- Printed worksheet (optional)
- Printed project brief
- Project files: l04-mini-game-project-kit.md, l04-guessing-game-example.py, l04-teacher-demo-script.md

Timed lesson plan (60 min)

Time	Stage	What happens
5 min	**Warm-up game**	Two truths and an AI lie: teams vote which code statement is the AI's mistake, and how to test it.
5 min	**Watch + pick (poll)**	Play the project video. Quick poll: which mini game will you build? Learners write the one-sentence idea.
5 min	**Live demo**	Teacher plays the example guessing game, catches or verifies the randint explanation, and writes one log entry live.
5 min	**Plan + AI review (pairs)**	Pairs swap pseudocode and review each other's plan, then each asks the AI to review it (no code).

20 min	**Build**	Learners build their game, logging every AI chat. Halfway check-in: thumbs up if the core works. Honest log entry Prompt Lab and the sort for anyone waiting for help.
6 min	**Test & peer feedback**	Breakout pairs play each other's games, try to break them, and give two stars and a wish.
3 min	**Quick competition**	Project route order race — first three correct win a shout-out.
6 min	**Show-and-tell**	Volunteers demo their game and explain one line the class picks, plus one AI mistake from their log.
5 min	**Probe + submit**	New situation: the "better version", then submit the game, log, tests and reflection.

Unplugged warm-up

Two truths and an AI lie (5 min): read three statements about code ("print shows text", "range(3) gives 0, 1, 2", "randint(1, 20) never gives 20"). Teams vote which is the AI's lie, then explain how they could test it. Link to the log: the test is what proves it.

Teaching the idea

This is the Module 1 project. Each learner builds a small game in Python or Scratch using the routine from Lessons 1–3: plan in pseudocode, AI review, build small, debug with bug reports, test like a player. The key deliverable alongside the game is the what-AI-did log: what they asked, what it said, how they checked, the result and what they changed. Reward honesty — catching an AI mistake is worth credit. The final check is that the learner can explain every line; AI-written code they cannot explain does not count. Keep the age rules and privacy rule as before.

Analogy: The what-AI-did log is like the "method" section of a science experiment: it shows how you got your result, including what went wrong.

Live demo: Show a finished example: a guessing game in the Python Lab (secret = random.randint(1, 20), a while loop, too high / too low, a guess counter). 1) Play it once. 2) Ask the AI helper: "Explain random.randint(1, 20) simply. Don't rewrite it." If it says 20 is not included, test it by printing the secret in a loop of 30 and show 20 appear (or run it until it does) — "Caught it!". If it is correct, still test it: "We trust, then verify." 3) Write the log entry live on the shared screen using the four parts. 4) Ask a volunteer to point at any line; explain it in one sentence.

Questions to ask

Ask	Listen for
What four parts make a good what-AI-did log entry?	What I asked, what it said, how I checked, the result (and what I changed).
Why does catching an AI mistake earn credit?	It shows you checked and understood, which is the real skill.

What should you do if AI offers a complete game?	Decline, ask about one feature, build it yourself and log the help.
How do you know a line is really yours?	You can explain what it does and why it is there.

Differentiation

Support
Offer the quiz game from Lesson 3 as a base: add two questions and a rating ("Cricket legend!" for 5/5). Log entries can be oral, recorded by the teacher.

Stretch
Add a "play again?" loop, a high score, or levels of difficulty. Ask the AI to suggest three test cases you have not thought of — and run them.

Common misconceptions

Misconception	How to fix it
Using AI less makes a worse project	Using AI well (explain, debug, review) and logging it honestly is what scores highly.
The log is just paperwork	The log is evidence of your thinking and checking — it is a rubric criterion.
If it runs, it is finished	It is finished when it is tested and you can explain every line.
randint works like range	randint(1, 20) includes 20; range(1, 20) stops at 19.

Assessment

Project rubric (also on the project brief):

Criterion	1 · Beginning	2 · Developing	3 · Secure	4 · Excellent
Game works	Does not run	Runs but has a major bug or no ending	Runs with a clear goal, score or ending	Polished, fun and handles silly inputs
Planning & testing	No plan or tests	A plan or tests, but incomplete	Pseudocode plan and 5 tests with results	Plan improved after AI review; tests include edge cases and fixes
Honest AI log	No log	Log is vague ("AI helped")	3+ entries with asked, said, checked, result	Detailed entries, including AI mistakes caught and how
Understanding	Cannot explain the code	Can explain some lines	Can explain every line	Explains every line and why it was written that way

Exit ticket
What is one thing the AI did for your game, and how did you check it?

Homework
Teach someone at home to play your game. Ask them to try to break it, fix one thing they find, and add a final log entry.

Answer key — Kitabash activities

1. Submit: my AI-assisted mini game (probe · project)

2. Prompt Lab: an honest log entry (perform · ai lab)

Checklist: What you asked · What the AI said · How you checked · The result · What you changed yourself

Strong example:

```
I asked: "Why does my guessing game never say Too low?" It said my if and elif were in the wrong order and suggested swapping them. I checked by running the game and guessing 3 when the secret was 15. It worked — the AI was right. I changed the order of the two lines myself.
```

3. Honest log or not OK? (perform · categorize)

Honest and OK: “AI explained randint; I tested it and it was wrong.”, “AI suggested swapping two lines; I did it and tested it.”, “Line 12 idea came from AI — I can explain it.”
Not OK: Pasting a whole AI-written game and logging nothing, Leaving out the times the AI was wrong to look clever, Saying “I wrote all of it” when AI wrote the loop

4. The project route (perform · ordering)

Order: Write the idea and pseudocode → Ask the AI to review the plan → Build the core and run it → Add the score and ending → Run five player tests → Explain every line and submit

5. New situation: the “better version” (probe · multiple choice)

Answer: Say no thanks, ask for a hint on just one feature, and build it herself

Answer key — worksheet

1. **Key words:** mini game = A small game with a clear goal and ending. · what-AI-did log = A record of each time you used AI: asked, said, checked, result. · player test = Playing your game with chosen inputs to check it works. · random.randint(a, b) = Python: a random whole number from a to b, both included.
2. **Prompt Lab: an honest log entry:** Include: What you asked; What the AI said; How you checked; The result; What you changed yourself. Example: I asked: "Why does my guessing game never say Too low?" It said my if and elif were in the wrong order and suggested swapping them. I checked by running the game and guessing 3 when the secret was 15. It worked — the AI was right. I changed the order of the two lines myself.
3. **Honest log or not OK?:** Honest and OK: “AI explained randint; I tested it and it was wrong.”, “AI suggested swapping two lines; I did it and tested it.”, “Line 12 idea came from AI — I can explain

it.” · Not OK: Pasting a whole AI-written game and logging nothing, Leaving out the times the AI was wrong to look clever, Saying “I wrote all of it” when AI wrote the loop

4. **The project route:** 1 Write the idea and pseudocode · 2 Ask the AI to review the plan · 3 Build the core and run it · 4 Add the score and ending · 5 Run five player tests · 6 Explain every line and submit
5. **New situation: the “better version”:** B. Say no thanks, ask for a hint on just one feature, and build it herself — Code she cannot explain is not hers. She can still learn from it: ask about one feature, then build it herself and log the help.
6. **My plan:** A clear plan that covers every requirement.
7. **Checklist:** All requirements met.
8. **Two stars and a wish:** Kind, specific feedback and an improvement.

Kitabash original curriculum. Product names (ChatGPT, Gemini, Claude, Copilot, Canva, NotebookLM, Perplexity, Teachable Machine) belong to their owners; Kitabash is not affiliated with or endorsed by them. Tools and age rules change: check each tool’s current terms before class.